Lab 2 — Agile Backlog & Sprint Simulation

OpenAPI Mock Server Generator

Pranav Kalla — PES1UG24AM197

Jira Space Key: OMSG

1. README — Project Overview

This document compiles the Agile backlog, sprint execution, and reporting deliverables for Lab 2, built on top of the Functional and Non-Functional Requirements defined in Lab 1.

Project Summary

The OpenAPI Mock Server Generator is a developer-productivity tool that ingests OpenAPI 3.0 specifications (YAML/JSON), generates dynamic mock REST endpoints with schema validation, and simulates configurable response latency — allowing API Developers and QA Engineers to test against a realistic mock API before the real backend exists.

Actors: API Developer, QA Engineer

Backlog was derived directly from the 5 Functional Requirements (FR-001–005) and 2 Non-Functional Requirements (NFR-001–002) specified in Lab 1, organized into 5 Epics and 16 User Stories, executed across 2 one-week sprints in Jira.

2. Jira Lab 2 Report

From Requirements to Epics

Each Epic groups related Functional Requirements from Lab 1. Non-Functional Requirements (NFR-001, NFR-002) were folded into a dedicated Epic (Epic 5) since they represent cross-cutting performance and reliability concerns rather than user-facing features.

Epic	Jira ID	Source Requirement(s)	Priority
Specification Ingestion & Validation	OMSG-1	FR-001	High
Mock Endpoint Generation	OMSG-2	FR-002	High
Request Validation	OMSG-3	FR-003	High
Response Configuration	OMSG-4	FR-004, FR-005	Medium
Performance & Reliability	OMSG-5	NFR-001, NFR-002	High

Epics and User Stories

Epic OMSG-1: Specification Ingestion & Validation (Priority: High)

Enable the API Developer to import, parse, and validate OpenAPI 3.0 specs before mock generation.

ID	User Story	Pts	Priority
OMSG-6	As an API Developer, I want to upload an OpenAPI 3.0 spec file (YAML/JSON) or import via URL, so that the system can generate mocks from it.	5	High
OMSG-7	As an API Developer, I want the system to validate the spec's structural conformance to OpenAPI 3.0, so that I know it's parseable before proceeding.	8	High
OMSG-8	As an API Developer, I want descriptive error messages with the location of syntax/schema errors, so that I can quickly fix a malformed spec.	3	Medium

Epic OMSG-2: Mock Endpoint Generation (Priority: High)

Automatically generate live, schema-accurate mock REST endpoints from the parsed spec.

ID	User Story	Pts	Priority
OMSG-9	As an API Developer, I want the system to auto-create mock endpoints for every path/method in my spec, so that I don't configure each one manually.	8	High
OMSG-10	As a QA Engineer, I want mock endpoints to return randomized JSON responses matching the declared schema, so that I can test against realistic data.	8	High
OMSG-11	As a QA Engineer, I want undefined paths to return 404, so that I can verify my client handles unknown endpoints correctly.	2	Medium

Epic OMSG-3: Request Validation (Priority: High)

Validate incoming requests against schema and surface errors.

ID	User Story	Pts	Priority
OMSG-12	As a QA Engineer, I want incoming requests validated against the declared schema (params, headers, body), so that I catch contract violations early.	8	High

ID	User Story	Pts	Priority
OMSG-13	As a QA Engineer, I want a 400 response with detailed validation errors on an invalid request, so that I know exactly what's wrong.	3	High
OMSG-14	As a QA Engineer, I want to view a log of requests/responses (including rejected ones), so that I can review my test session history.	5	Medium

Epic OMSG-4: Response Configuration (Priority: Medium)

Let developers customize mock behavior — latency and overrides — to emulate real-world conditions.

ID	User Story	Pts	Priority
OMSG-15	As an API Developer, I want to set fixed or randomized latency globally or per-endpoint, so that I can simulate real network conditions.	5	Medium
OMSG-16	As an API Developer, I want to override an endpoint's response with a custom payload, so that I can simulate specific business scenarios.	5	Medium
OMSG-17	As an API Developer, I want to force a specific status code (e.g., 401, 429, 500) on an endpoint, so that I can test my client's error handling.	2	Medium

Epic OMSG-5: Performance & Reliability (Priority: High)

Ensure the mock server is fast, stable, and secure under load.

ID	User Story	Pts	Priority
OMSG-18	As an API Developer, I want the mock server to sustain ≥1,000 req/s, so that it doesn't bottleneck CI pipelines.	13	High
OMSG-19	As a QA Engineer, I want configured latency to stay accurate within ±10ms under peak load, so that my results are trustworthy.	8	High
OMSG-20	As an API Developer, I want malformed or oversized (>5MB) spec uploads rejected gracefully, so that the server never crashes.	5	High
OMSG-21	As an API Developer, I want each mock instance isolated with sanitized inputs, so that no injection or data leak occurs.	8	High

Total backlog size: 5 Epics, 16 User Stories, 84 story points.

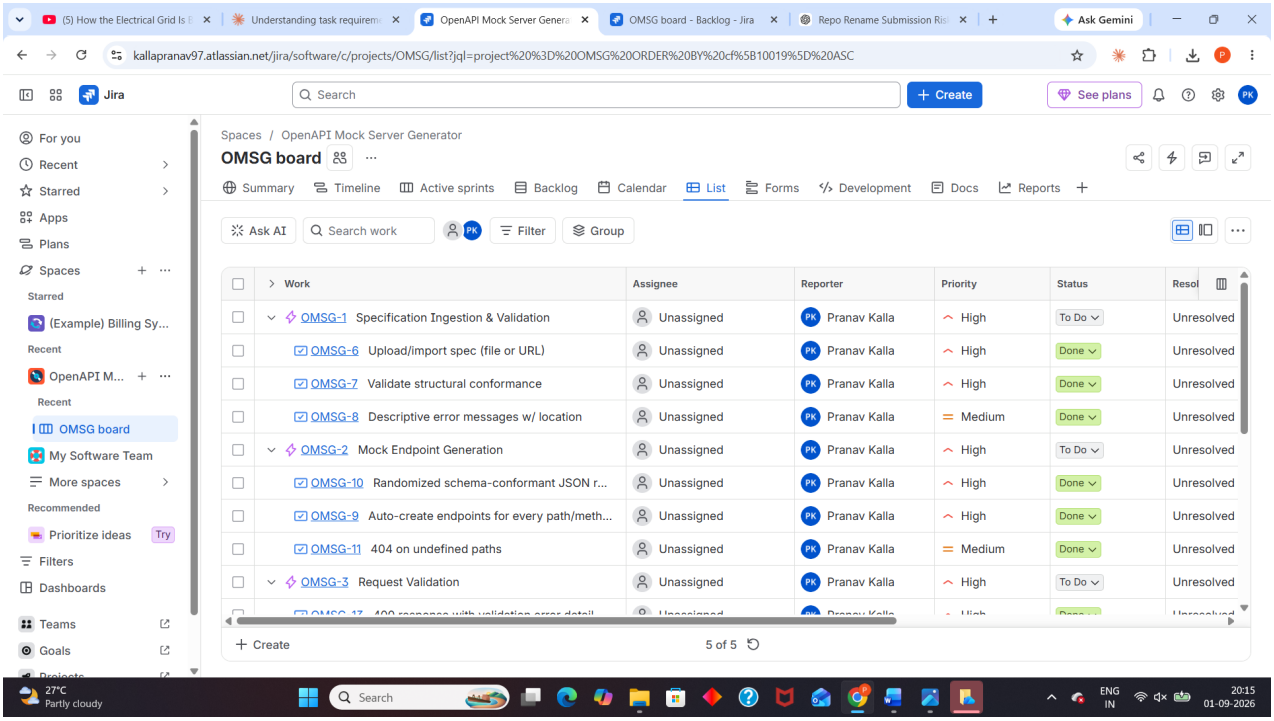
Estimation Approach (Planning Poker)

Story points were assigned using the Fibonacci sequence (1, 2, 3, 5, 8, 13), reflecting effort, complexity, and uncertainty rather than raw time. Parsing/validation and load-related stories (OMSG-7, OMSG-9, OMSG-10, OMSG-12, OMSG-18, OMSG-19, OMSG-21) were scored higher due to genuine technical uncertainty. Simple routing/status-code behaviours (OMSG-11, OMSG-17) were scored lowest. Priority was carried over from the FR/NFR priority levels in Lab 1, with a few supporting stories (OMSG-8, OMSG-14) set to Medium since they refine rather than define core functionality.

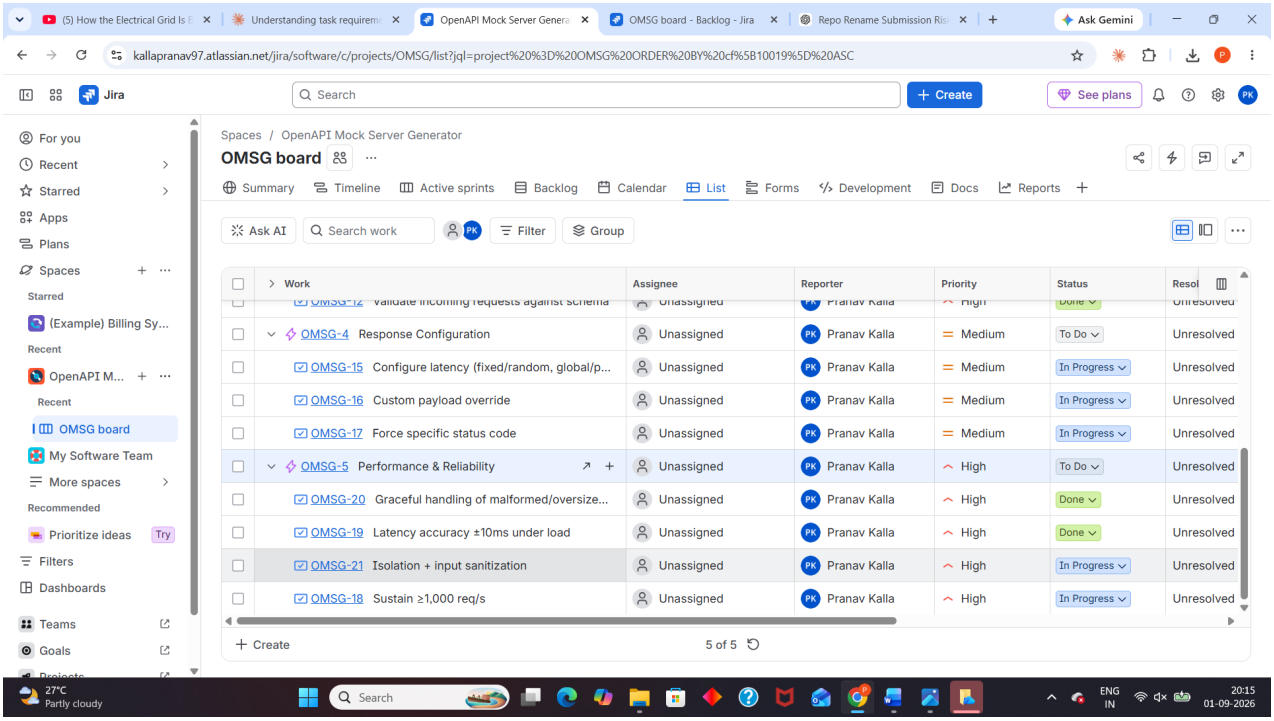
Jira Board Setup

A Scrum-type space (“OpenAPI Mock Server Generator”, key OMSG) was created in Jira with a Backlog, Board, and Reports view. All 5 Epics were created first, followed by 16 User Stories linked to their respective Epics via the Epic field.

List view — all 5 Epics with their linked User Stories nested underneath (part 1 of 2):



Epics & Stories — part 1



Epics & Stories — part 2

3. Sprint Summary

Per the lab's timeline, both sprints were planned and executed within a single session. Each sprint was configured in Jira with a 1-week duration (Sep 1 – Sep 8) as required, with both sprints' full cycles — planning, execution, and completion — carried out back-to-back on the same day. Two 1-week sprints were run to deliver the 16-story, 84-point backlog.

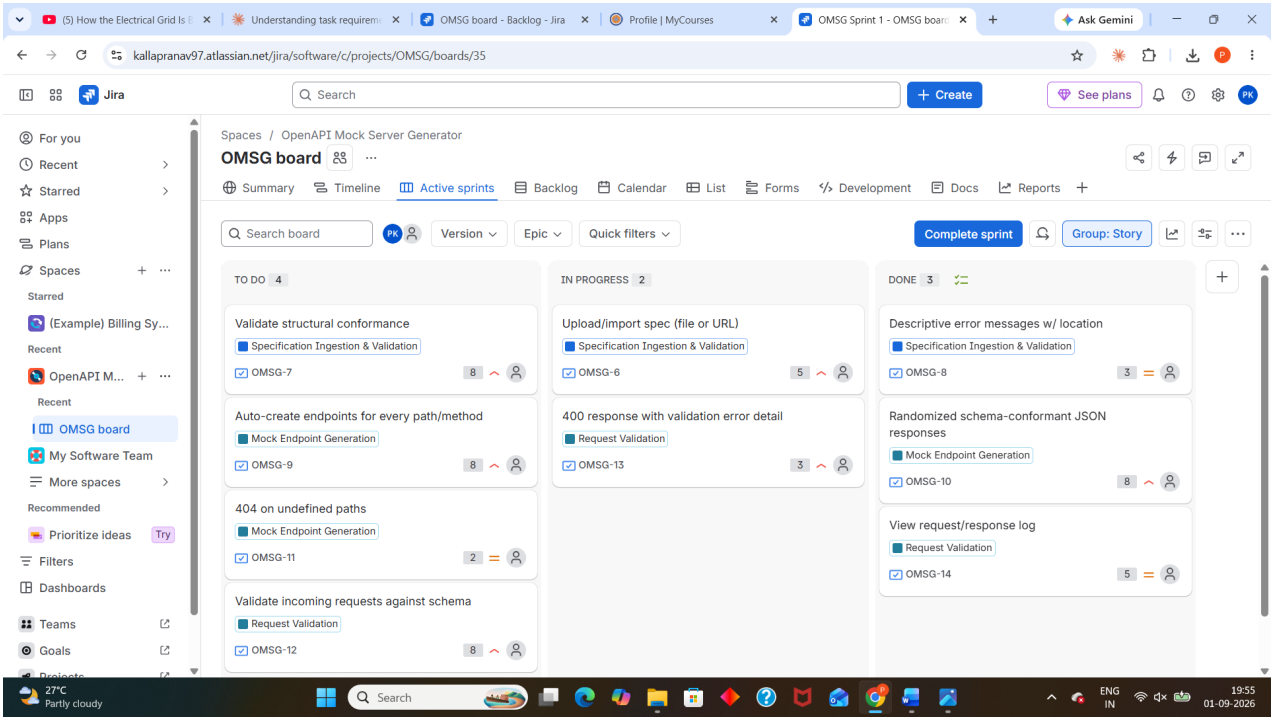
Sprint 1 (Sep 1 – Sep 8)

Scope: Epics 1–3 (Specification Ingestion & Validation, Mock Endpoint Generation, Request Validation) — the core ingestion-to-validation “walking skeleton” of the system.

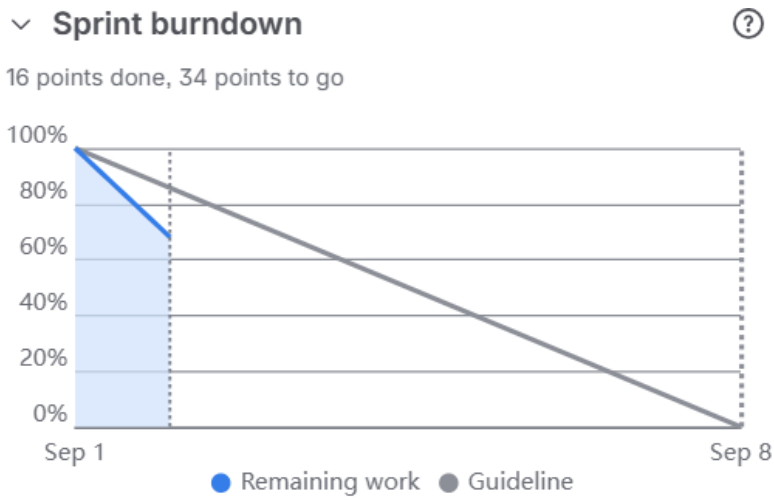
Story	Points
OMSG-6 Upload/import spec	5
OMSG-7 Validate structural conformance	8
OMSG-8 Descriptive error messages	3
OMSG-9 Auto-create endpoints	8
OMSG-10 Randomized schema-conformant responses	8
OMSG-11 404 on undefined paths	2
OMSG-12 Validate incoming requests	8
OMSG-13 400 response with validation error detail	3
OMSG-14 View request/response log	5
Sprint 1 total	50

Stories were progressed through To Do → In Progress → Done over the sprint, with progress comments logged on each card as work advanced.

Sprint 1 board (mid-sprint state):

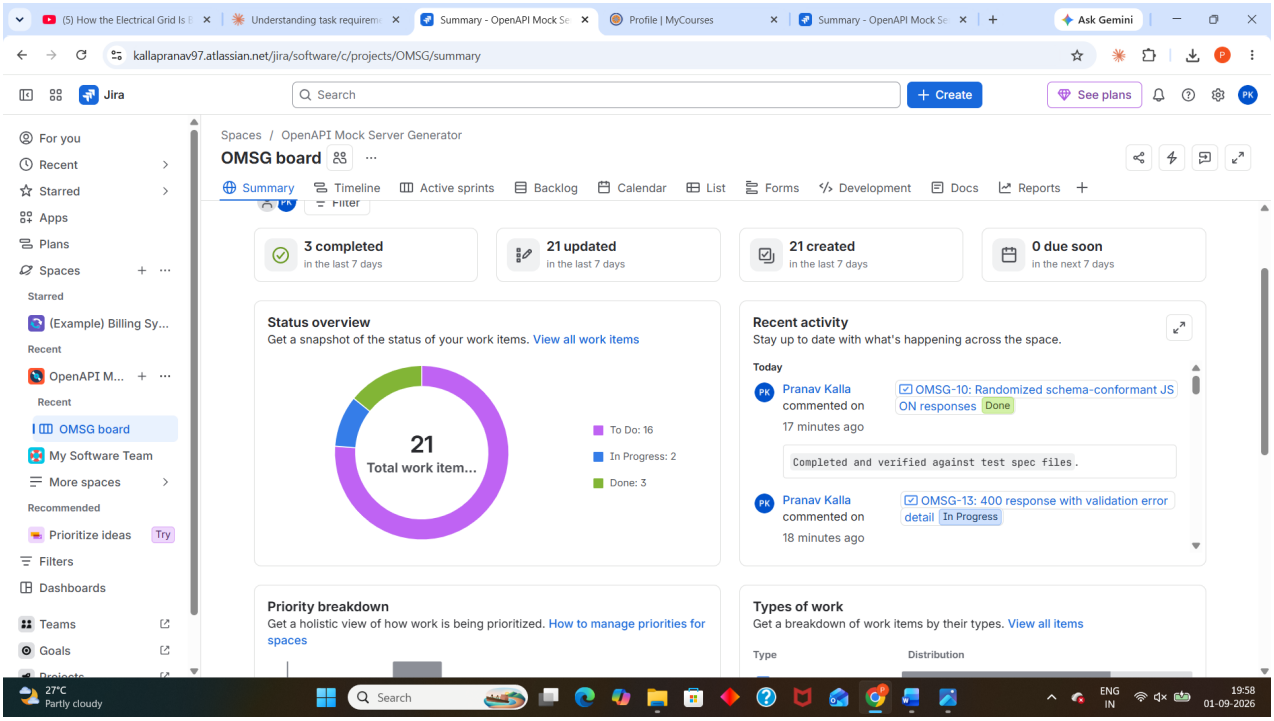


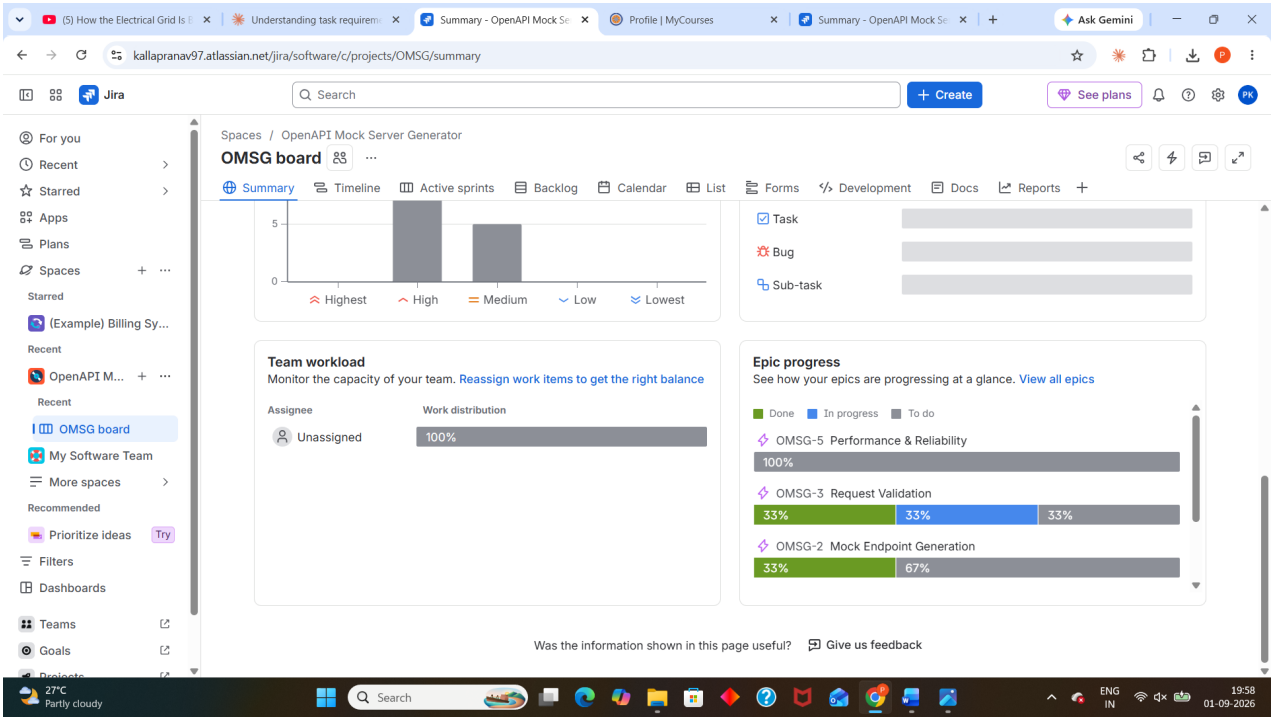
Sprint 1 burndown chart:



At the point captured, 16 of 50 points were complete, tracking close to the ideal guideline.

Space summary after Sprint 1:





At sprint close, any incomplete Sprint 1 stories rolled forward into Sprint 2 per standard Scrum practice.

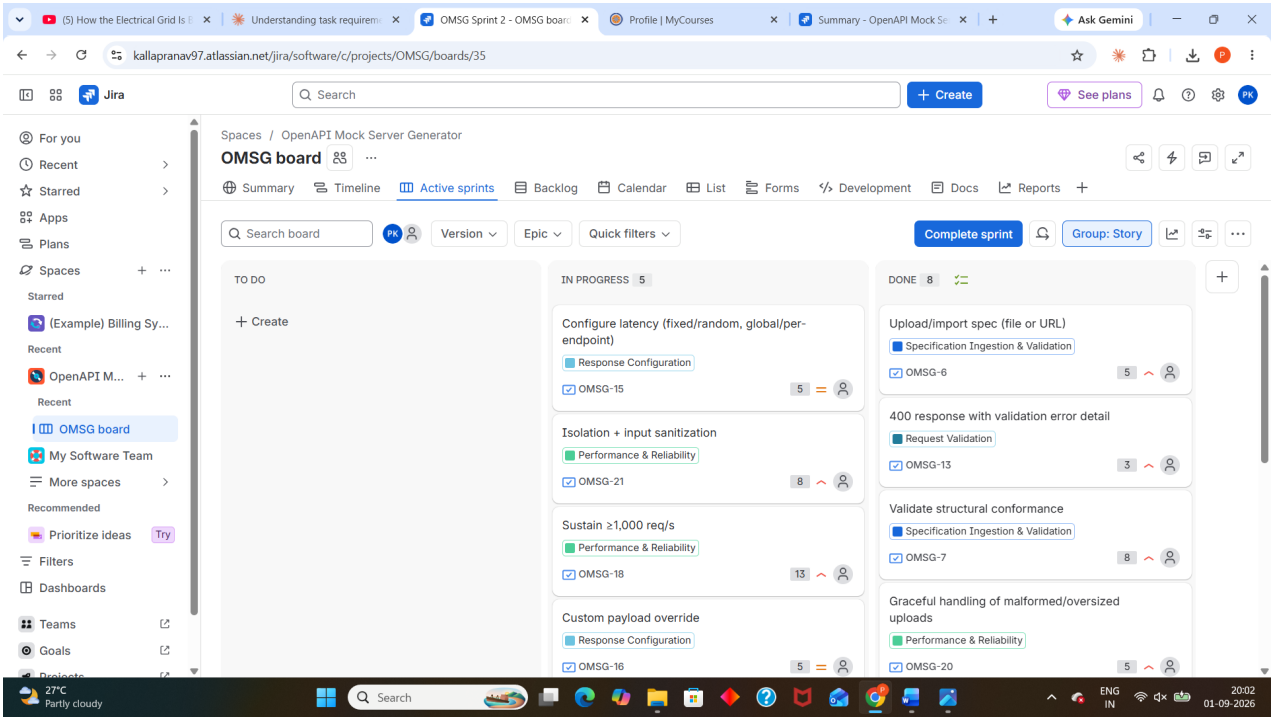
Sprint 2 (Sep 1 – Sep 8)

Scope: Epics 4–5 (Response Configuration, Performance & Reliability) plus carry-over work from Sprint 1. Sprint 2 was planned and started immediately after Sprint 1 was completed, both within the same lab session, so both sprints share the same 1-week window (Sep 1 – Sep 8) as configured in Jira.

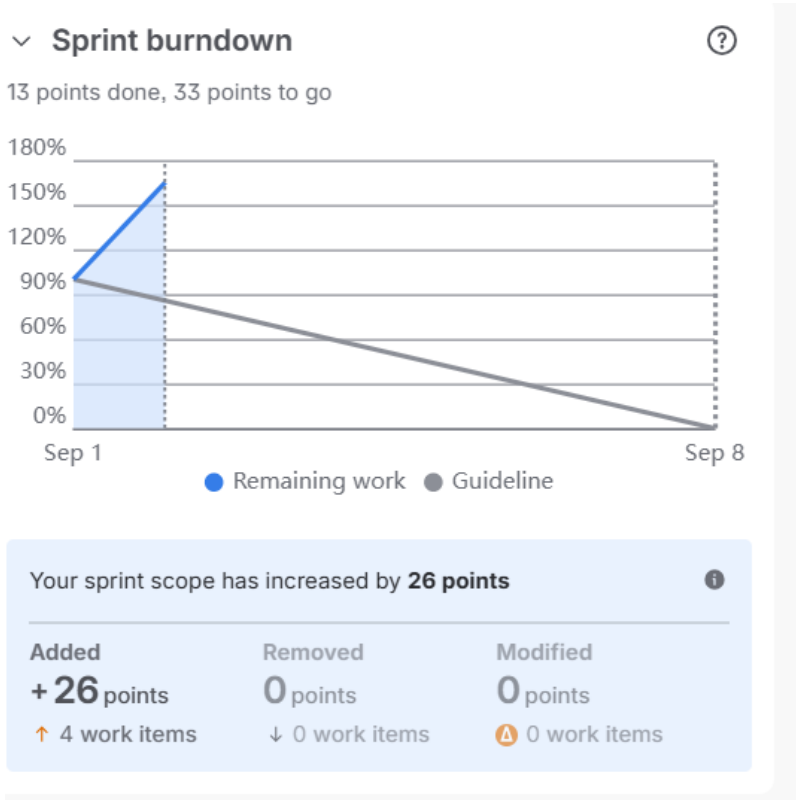
Story	Points
OMSG-15 Configure latency	5
OMSG-16 Custom payload override	5
OMSG-17 Force specific status code	2
OMSG-18 Sustain ≥1,000 req/s	13
OMSG-19 Latency accuracy ±10ms	8
OMSG-20 Graceful handling of malformed uploads	5
OMSG-21 Isolation + input sanitization	8
New scope total	46

Because unfinished Sprint 1 items carried forward, Sprint 2's total scope increased by 26 points (4 work items) over its initial plan — a realistic reflection of estimation variance.

Sprint 2 board:

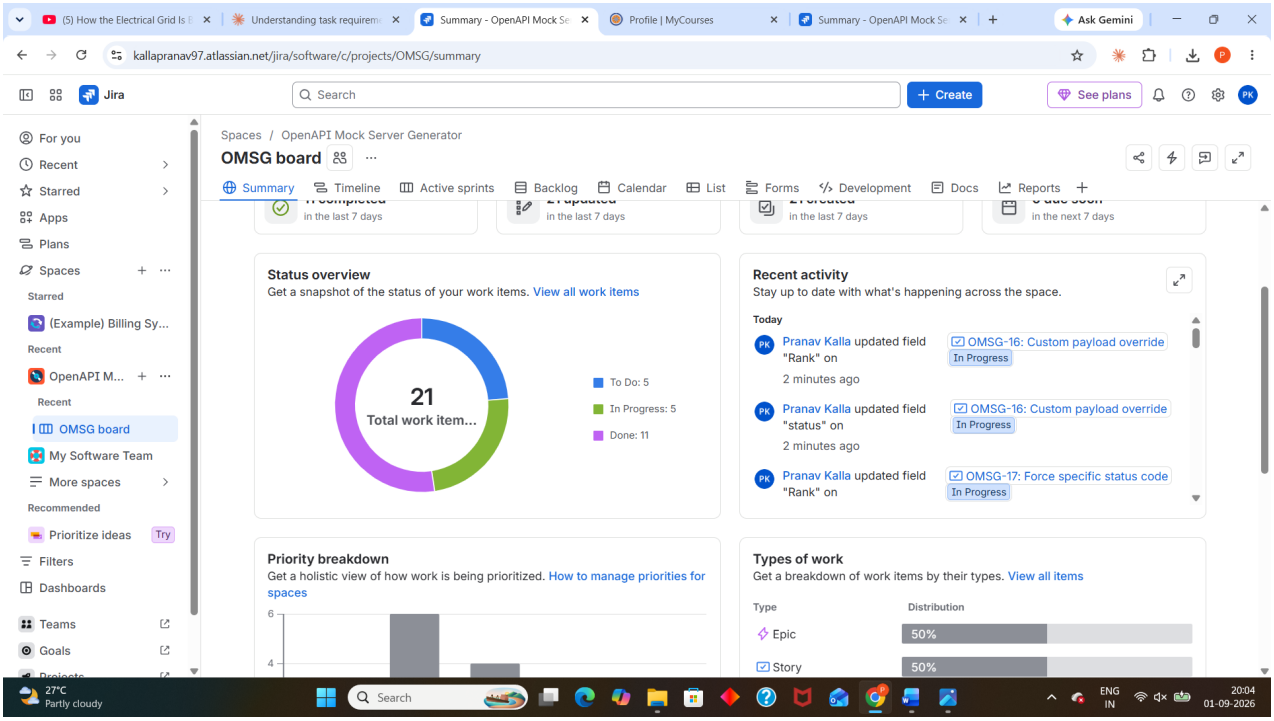


Sprint 2 burndown chart:



At the point captured, 13 of 46 points were complete, with scope growth clearly visible as the early spike in the burndown line.

Space summary after Sprint 2:



By the end of Sprint 2, 11 of 21 total work items were marked Done, with the remainder split between In Progress and To Do.

Reflection

• **Scope carry-over is normal, not a failure.** Sprint 1's unfinished items rolling into Sprint 2 (visible as the scope-increase spike on the Sprint 2 burndown) reflects how real teams handle incomplete work — it gets re-planned, not discarded.

• **High-uncertainty stories** (e.g., OMSG-18 “Sustain ≥1,000 req/s”, 13 points) deserved their higher estimate — performance and reliability work carries more inherent risk than straightforward CRUD-style stories, which the Fibonacci scale is designed to capture.

• **Grouping NFRs into their own Epic (Epic 5)** made it easier to track cross-cutting quality attributes separately from user-facing functionality, rather than losing them inside feature Epics.

• **Burndown guidelines are a signal, not a verdict** — tracking below or above the guideline line early in a sprint is useful for spotting estimation or capacity issues, but the real value came from the discussion it prompted about why scope shifted between sprints.

4. Submission Checklist

Mapped directly against the deliverables specified for Lab 2.

#	Deliverable	Status
1	Screenshots of Epics & User Stories created (your scenario)	Done
2	SS of created sprint (To Do, Progress, Done)	Done
3	Burndown Chart, Summary	Done
4	2 Sprints (1 week each)	Done

Backlog scale delivered

Metric	Value
Epics	5
User Stories	16
Total story points	84
Sprints completed	2 (1 week each)